Active-Learning-Assisted Candidate Screening for Flexible Job Shop Scheduling

Xianyi Xie, Liwei Liu
School of Computer and Artificial Intelligence, Beijing Technology and Business University
Beijing 100048, China
Emails: 2431062085@st.btbu.edu.cn; 2431062177@st.btbu.edu.cn
Corresponding author: Xianyi Xie

***Abstract*—Flexible Job Shop Scheduling (FJSP) couples routing and sequencing decisions, making repeated exact evaluation expensive. We study a candidate-screening workflow in which a heuristic ranks machine assignments, time-limited MIP solves provide calibration labels, and a Random Forest (AL-RF) ranks an independent test pool. In the original two-seed Brandimarte MK01–MK10 evaluation, AL-RF has the lowest mean best feasible makespan among five methods given the same five final verifications. Its 30 calibration calls add substantial cost. A five-seed comparison counts those calls and compares AL-RF's best calibration or final solution with 35 heuristic-ranked verifications. Each method performs better on five instances; AL-RF does not improve the overall mean ($p = 0.943$). The distinction between final and total calls is central to assessing the learned screener.**

Keywords: flexible job shop scheduling; random forest; surrogate model; decomposition; heuristic labeling; solver budget

I. Introduction

In a flexible job shop, an operation can run on more than one machine, often with a different processing time on each. Given jobs with fixed operation order, scheduling must assign every operation to an eligible machine and sequence operations sharing that machine. We minimize makespan, the completion time of the last job.

The coupling of routing and sequencing is the central computational difficulty of FJSP. Routing determines which operations compete for each machine, while sequencing resolves those conflicts. Mixed-integer programming (MIP) formulations model both decisions, and solving larger instances can be difficult [26]. Metaheuristics, including the tabu search of [23], offer an alternative. Here we ask whether solutions to a small set of assignments can guide which other assignments receive MIP verification.

Learning-based scheduling includes dispatching policies trained with reinforcement learning or graph neural networks [35], [30], [2]. Surrogate-assisted optimization instead uses a predictor to guide expensive evaluations [17]. Our RF ranks assignments externally rather than being embedded in the MIP [22].

This paper investigates whether lightweight, interpretable features of a machine assignment can provide useful guidance for screening candidate assignments under a fixed verification budget. A machine assignment is represented by workload, operation-count, machine-usage, imbalance, dispersion, and maximum job-time features. A fast heuristic sequencer generates low-cost proxy labels, a Random Forest surrogate scores unseen assignments, and a time-limited fixed-assignment MIP verifies the same number of candidates for every scorer.

Our research is guided by four questions:

- **RQ1.** Can aggregate assignment features improve final Top-K feasible incumbent quality under a fixed verification budget?
- **RQ2.** Under the same final verification budget, does a learned RF surrogate improve on simpler scoring rules?
- **RQ3.** How much does the relative performance vary across the Brandimarte instances?
- **RQ4.** Does the method still help when calibration calls count toward the solver budget?

Our contributions are:

- An active-learning-assisted screening workflow that fits an RF to a small, deliberately selected set of MIP-verified training assignments before ranking unseen machine assignments.
- A common-pool evaluation with fixed final Top-K verification and a separate five-seed comparison that counts calibration calls.
- Evidence that the lower mean in the original five-verification comparison does not carry over to the equal-call comparison with a direct heuristic.

II. Related Work

A. FJSP: Problem, Benchmarks, and Foundations

FJSP was formalized by [6] as a generalization of the classical job shop. The standard benchmark suites—Brandimarte [4], Hurink *edata*/*rdata*/*vdata* [15], Dauzère-Pérès and Paulli [8], Kacem [18], Fattahi [11], and others—have driven algorithm development for three decades. FJSPLib provides a centralized repository of instances and best-known lower/upper bounds across these suites. [28] provides a comprehensive textbook treatment. [34] classify FJSP solution methods into exact algorithms, heuristics, and meta-heuristics.

B. Exact, MIP, and CP Methods

Exact MIP formulations jointly model assignment and sequencing [26], [24]. While they can prove optimality on small

instances, their computational cost grows exponentially. Constraint Programming (CP) approaches, notably CP-SAT and IBM CP Optimizer, have demonstrated strong performance on FJSP benchmarks through interval variables and specialized scheduling constraints. However, both MIP and CP solvers still require substantial time on medium-to-large instances, motivating decomposition and heuristic approaches.

C. Metaheuristics and Hybrid Methods

The tabu search of [23] uses hierarchical neighborhoods to find strong schedules. Subsequent work includes genetic algorithms [27], knowledge-guided search for dual-resource variants [36], hybrid genetic-tabu search [21], variable neighborhood search [12], and particle swarm optimization hybrids [33]. [7] review AI techniques for job shop scheduling. These methods provide established alternatives to candidate screening.

D. Decomposition in Scheduling

Decomposition has a long history in scheduling. The shifting bottleneck procedure [1] iteratively solves single-machine subproblems. For FJSP, several works separate assignment and sequencing: [4] uses a hierarchical tabu search, and [33] hybridize particle swarm optimization for assignment with simulated annealing for sequencing. These methods use fixed heuristics for the assignment subproblem. Our work differs by *learning* an evaluation function for assignments from data.

E. Learning-Based Scheduling

End-to-end methods learn dispatching policies: [35] represent job shop states as disjunctive graphs; [30] extend graph-based control to FJSP; [32] use dual attention; and [25] address transportation constraints. [29] study stochastic FJSP. Our RF has a narrower role: ranking a fixed pool of assignments for MIP verification.

Learning-to-configure approaches use ML to accelerate existing solvers: [16] for algorithm configuration, [14] and [19] for branching in MIP, and [13] for node selection in B&B via graph convolutional networks. [20] survey end-to-end constrained optimization learning.

F. Surrogate-Assisted Combinatorial Optimization

Surrogate models replace expensive objective evaluations with cheap approximations. [17] surveys surrogate-assisted evolutionary computation, primarily for continuous optimization. Extensions to discrete and combinatorial problems are more recent. The key challenge in discrete settings is defining feature representations that preserve the ranking of decision alternatives under the true objective. Our work addresses this by designing structured aggregate features for the FJSP assignment space and validating ranking quality through top-K metrics.

G. Tree-in-MIP and Constraint Learning

Tree-based predictors can be embedded in mixed-integer models [31], [22], and optimization can also guide tree construction [3]. Our method does not embed the RF in an MIP: it ranks assignments externally, then uses a conventional sequencing model for verification. This distinction places the workflow within prediction-guided optimization [10].

H. Positioning

The workflow combines assignment features, heuristic labels, an RF ranking, and MIP verification. Its calibration calls are counted when comparing it with direct heuristic ranking.

III. Problem Formulation and Lower-Bound Motivation

A. Notation

Table I defines the notation. Each job follows a prescribed operation order, and each operation must be processed without interruption on one eligible machine.

TABLE I
NOTATION.

$J = \{1, \dots, n\}$	Set of jobs
$M = \{1, \dots, m\}$	Set of machines
$\mathcal{O} = \{(j,p)\}$	Set of all operations
$M_{j,p} \subseteq M$	Feasible machines for (j,p)
$p_{j,p,k}$	Processing time of (j,p) on k
$x_{j,p,k} \in \{0,1\}$	Assignment variable
$S_{j,p}, C_{j,p} \geq 0$	Start and completion times
$\delta_{(j,p),(j',p'),k} \in \{0,1\}$	Sequencing variable
$C_{\max}$	Makespan
H	Big-M constant ($\sum_{j,p,k} p_{j,p,k}$)

B. Full FJSP MIP Formulation

The standard MIP minimizes $C_{\max}$ subject to:

$$\sum_{k \in M_{j,p}} x_{j,p,k} = 1 \quad \forall (j,p) \in \mathcal{O} \tag{1}$$

$$C_{j,p} = S_{j,p} + \sum_{k \in M_{j,p}} p_{j,p,k}\, x_{j,p,k} \quad \forall (j,p) \tag{2}$$

$$S_{j,p+1} \geq C_{j,p} \quad \forall j, p < o_j \tag{3}$$

$$S_{j',p'} \geq C_{j,p} - H(1-\delta) - H(2 - x_{j,p,k} - x_{j',p',k}) \tag{4}$$

$$C_{\max} \geq C_{j,p} \quad \forall (j,p) \tag{5}$$

where (4) is enforced for all machine-sharing operation pairs with the complementary constraint $S_{j,p} \geq C_{j',p'} - H\delta - H(2 - x_{j,p,k} - x_{j',p',k})$.

C. Lower Bounds and Feature Motivation

Given a fixed assignment $\mathbf{x}$, the sequencing subproblem is:

$$\mathrm{SEQ}(\mathbf{x}): \min_{\delta, S, C} C_{\max} \text{ s.t. } (2) - -(5) \text{ with } \mathbf{x} \text{ fixed.} \tag{6}$$

Two classical lower bounds motivate our feature design:

Proposition 1 (Machine workload bound). For any feasible schedule under assignment $\mathbf{x}$: $C_{\max} \geq \max_{k \in M} \sum_{(j,p):x_{j,p,k}=1} p_{j,p,k}$.

Proposition 2 (Job path bound). For any feasible schedule: $C_{\max} \geq \max_{j \in J} \sum_p \sum_k p_{j,p,k}\, x_{j,p,k}$.

Proof. Proposition 1 holds because $\max_k \mathrm{load}_k$ is the earliest possible completion time of the bottleneck machine. Proposition 2 holds because each job's operations must be processed sequentially, so total job time is a lower bound on its completion time. □

These bounds justify machine workload and maximum job-time features as surrogate inputs. They are necessary but not sufficient: two assignments with similar aggregate loads can still have different sequencing difficulty because of precedence and machine interactions. The experiments therefore treat the lower bound as a baseline scorer and do not claim that the implemented feature vector captures the full fixed-assignment objective.

IV. Methodology

A. Aggregate Feature Representation

We represent an assignment $\mathbf{x}$ by the workload, operation-count, usage, path-time, and dispersion features implemented in the released experiment code and summarized in Table II. All features are computable from $\mathbf{x}$ alone and do not require solving the sequencing subproblem.

TABLE II
Feature groups implemented in the released scorer.

Group	Features	Motivation
A. Machine assignment	load_k, ops_k, used_k for each machine	Represent machine workload and assignment sparsity
B. Aggregate workload	number of used machines, maximum machine load, load imbalance, maximum operations per machine, load standard deviation	Lower-bound and balance indicators
C. Job-path summary	maximum total processing time of any job under the assignment	Proposition 2: critical job-path bound

B. Heuristic Sequencer and Labeling Procedure

We design a fast constructive heuristic sequencer to generate proxy labels $\tilde{C}_{\max}(\mathbf{x})$ for training. The sequencer maintains the ready set of operations whose predecessors have completed, applies the implemented dispatching rules, and returns the best resulting makespan with deterministic tie-breaking. The implementation performs no backtracking or insertion; its runtime is measured empirically rather than assigned an overly specific asymptotic bound.

Training and test pools are generated using different deterministic seeds and deduplicated within each pool. Labels are fitted from the training pool only. The supplementary audit saves the assignments and checks that training and test pools do not intersect before fitting.

C. Active-Learning Surrogate

RF-H is trained on cheap heuristic labels. For AL-RF, we sort the training assignments by these labels and select q evenly spaced rank positions across the full ranking, including its endpoints. Their time-limited MIP incumbent makespans train a separate Random Forest [5], which scores the independent test pool. Thus, the implementation uses one batch of deliberately selected labels, rather than iterative uncertainty-based acquisition. All feasible calibration incumbents are accepted. With fewer than five feasible labels, the scorer returns their mean; if none is available, all scores are $+\infty$ and the implementation's ordering resolves the ties.

D. Candidate Screening and Budget Accounting

The final comparison uses a common test pool. LB, RF-H, AL-RF, HeurSeq, and Random score exactly the same candidates and each submit K candidates to the fixed-assignment MIP. AL-RF has an explicit calibration cost of q MIP calls; this cost is reported separately from the final screening budget. All methods receive the same final K verifications, so the comparison answers whether active calibration improves the quality of the final shortlist.

Algorithm 1 Active-Learning Candidate Screening

1: **Input:** Instance, training pool $\mathcal{T}$, test pool $\mathcal{V}$, calibration size q, top-K
2: Generate heuristic labels on $\mathcal{T}$ and fit the initial RF-H model
3: Select q evenly spaced positions in the heuristic ranking of $\mathcal{T}$ and verify them with the fixed-assignment MIP
4: Fit AL-RF on feasible MIP incumbents; use their mean if fewer than five labels are available (all $+\infty$ if none)
5: **for** each of the five scorers s **do**
6: Score the common test pool $\mathcal{V}$
7: Select the same number K of lowest-scoring assignments
8: Verify selected assignments with the identical final MIP budget
9: **end for**
10: Report final and calibration incumbents separately; charge calibration and final solver calls

E. MIP-Based Sequencing Verification

Top-K candidates and calibration assignments are verified by solving $\mathrm{SEQ}(\mathbf{x})$ with Gurobi and a five-second limit per call. The supplementary audit saves incumbent, bound, status, gap, elapsed time and a validated schedule for every feasible solve. A time-limited incumbent is not a proof of optimality; reported quality therefore refers to feasible solutions under the declared budget.

V. Computational Experiments

A. Experimental Setup

Instances. We evaluate on Brandimarte MK01–MK10 [4], with 55–240 operations. Table III reports counts and average

eligible machines per operation recomputed from the benchmark files; reference BKS values are retained from the original development setup. MK06 has 10 machines. Its historical input declared 15, adding five unused machine feature groups; the supplementary audit examines this correction separately.

TABLE III
INSTANCE CHARACTERISTICS.

Instance	n	m	N	Avg. flex.	BKS
mk01	10	6	55	2.09	40
mk02	10	6	58	4.10	26
mk03	15	8	150	3.01	204
mk04	15	8	90	1.91	60
mk05	15	4	106	1.71	172
mk06	10	10	150	3.27	58
mk07	20	5	100	2.83	139
mk08	20	10	225	1.43	523
mk09	20	10	240	2.52	307
mk10	20	15	240	2.98	198

Implementation. Each run uses 200 training and 100 test assignments, requests 30 calibration labels, selects $K = 5$, and allows five seconds per MIP call. The original evaluation uses seeds 42 and 43; the equal-call comparison uses seeds 42–46. Both RF models use 100 trees and depth 12; minimum leaf sizes are 2 for AL-RF and 5 for RF-H. The new runs use an Intel Core i5-14600KF, Gurobi 10.0.1 and scikit-learn 1.6.1. Gurobi retains its default automatic thread setting; RF fitting uses one worker. The original summaries and new run records are kept separate.

Baselines. We compare scorers under a common candidate pool and verification budget:

- **LB**: $\max\{\max_k \text{load}_k, \max_j \text{jobTime}_j\}$.
- **RF-H**: RF trained only on heuristic sequencing labels from the training pool.
- **AL-RF**: RF trained on the small MIP-verified calibration set described above.
- **HeurSeq** and **Random**: direct heuristic and seeded random rankings of the same test pool.

Metrics. The original table reports the mean best final feasible incumbent over two seeds. The new comparison counts AL-RF's best calibration or final incumbent against the best of 35 heuristic-ranked test candidates. We average five seeds within each instance, then apply a two-sided paired Wilcoxon signed-rank test [9]; differences below 10^{-3} count as ties. Calls without a valid incumbent remain charged but contribute no solution.

B. Active-Learning Development Evaluation

Table IV and Fig. 1 report the two-seed-per-instance development evaluation. Each row averages two independent runs with the same candidate-pool size, calibration size, final K, and five-second MIP limit. AL-RF has the lowest or tied-lowest mean in six of ten instances and the lowest overall mean across the 20 instance–seed runs. Its mean best feasible makespan is 211.7, compared with 214.2 for RF-H, 214.6 for LB, 212.8 for HeurSeq, and 217.9 for Random. Relative to

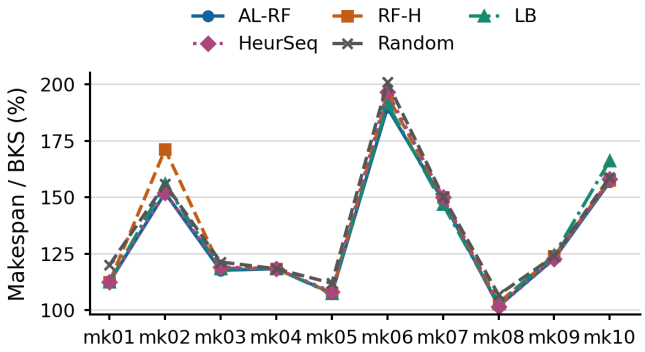


Fig. 1. Original two-seed development results, normalized by each instance's BKS. Each method receives five final verifications; AL-RF calibration calls are excluded. Lower is better.

HeurSeq, gains occur on MK03, MK05–MK07 and MK10, while HeurSeq is better on MK08 and MK09. These results motivate examining calibration cost, rather than establish an equal-total-budget advantage.

TABLE IV
ORIGINAL FIVE-SECOND SCREENING RESULTS (TWO-SEED MEANS; EQUAL FINAL TOP-5 CALLS, EXCLUDING CALIBRATION COST).

Instance	AL-RF	RF-H	LB	HeurSeq	Random
mk01	45.0	45.0	45.0	45.0	48.0
mk02	39.5	44.5	40.5	39.5	40.5
mk03	240.0	242.0	241.5	243.0	247.5
mk04	71.0	71.0	71.0	71.0	71.0
mk05	185.0	185.0	184.5	186.0	192.5
mk06	110.0	113.0	111.0	114.0	116.5
mk07	206.5	208.5	204.0	208.5	209.0
mk08	532.5	540.0	537.0	531.5	558.5
mk09	377.0	380.5	382.0	376.5	380.5
mk10	310.5	312.0	329.0	313.0	314.5

The original release contains summary outcomes but not all calibration incumbents. We therefore run a separate equal-call comparison rather than infer those solutions from the original table. AL-RF retains its best solution from 30 calibration and five final calls; HeurSeq verifies 35 test-pool assignments. This controls requested solver calls, not end-to-end wall-clock time.

Across the ten development instance means, AL-RF wins five, ties three, and loses two against HeurSeq. The paired signed-rank test gives $p = 0.094$; the small mean difference is not statistically significant.

Equal-call comparison. Table V reports a separate five-seed experiment. AL-RF may retain its best solution from 30 calibration and five final calls; HeurSeq-35 verifies 35 test-pool assignments. Each entry is the mean best feasible makespan over seeds 42–46. The training and test pools are disjoint.

AL-RF wins on five instances and HeurSeq-35 on five; the paired signed-rank test gives $p = 0.943$. Their mean charged optimization times are 162.9 and 149.0 seconds per run, respectively. Twelve invalid incumbents are excluded from

TABLE V
FIVE-SEED, 35-CALL COMPARISON. LOWER MAKESPAN IS BETTER.

Instance	AL-RF	HeurSeq-35	Difference
mk01	42.6	44.8	−2.2
mk02	40.2	40.8	−0.6
mk03	228.0	233.0	−5.0
mk04	67.8	69.0	−1.2
mk05	186.2	186.0	+0.2
mk06	106.2	104.0	+2.2
mk07	205.6	206.6	−1.0
mk08	529.4	525.0	+4.4
mk09	368.2	367.4	+0.8
mk10	312.8	306.8	+6.0
Mean	208.70	208.34	+0.36

TABLE VI
FULL FJSP CP-SAT REFERENCE, SEEDS 42 AND 43.

Instance	Seed 42	Seed 43	Mean gap to BKS
mk06	60.0	59.0	2.6%
mk10	233.0	231.0	17.2%

the results but their calls remain charged. The comparison controls requested calls, not total wall-clock time.

Full FJSP reference. To put the harder instances in context, Table VI gives results from a standard CP-SAT model with a 175-second deadline and one worker. This model optimizes routing and sequencing jointly; the screening methods solve only fixed-assignment sequencing problems.

VI. DISCUSSION

A. Role of the Surrogate

The models receive workload, assignment-usage, operation-count, imbalance, dispersion, and maximum job-time summaries. AL-RF is fitted directly to a small set of MIP-verified incumbents sampled across the heuristic ranking. Its role is to rank independent test assignments; this experiment does not establish a causal interpretation of feature importance.

B. Calibration Cost and Search Quality

AL-RF has a lower historical Top-5 mean, but its calibration uses 30 additional calls. When both methods receive 35 calls, AL-RF improves the instance mean on five cases and worsens it on five. A scheduler choosing between the methods would need to account for those calls, not only the final shortlist.

C. Instance-Level Variation

The five-seed comparison shows gains on MK01–MK04 and MK07, but losses on the other five instances. MK10 has the largest mean loss for AL-RF, at 6.0 makespan units; MK03 has its largest gain, at 5.0. These differences describe where this calibration helped, without identifying a structural cause.

D. Practical Implications

The workflow produces a shortlist of machine assignments for fixed-assignment MIP checks. Its calibration runs can themselves produce useful schedules, so their incumbents should be retained. Heuristic ranking remains a strong low-cost control, and neither method's time-limited incumbent is a proven optimum.

E. Limitations and Future Work

Aggregate features omit explicit precedence interactions, and five-second solves can return nonoptimal incumbents or no solution. The original table uses two seeds and the new budget comparison five; both use one benchmark family, and the added seeds are not an independent holdout. Equal call limits also leave differences in preprocessing and actual solver time. Broader benchmarks and end-to-end time budgets remain future work.

VII. CONCLUSION

We have presented an active-learning-assisted candidate-screening workflow for Flexible Job Shop Scheduling. An RF fitted to a small set of MIP-verified assignments ranks a common test pool. Across Brandimarte MK01–MK10 with two seeds per instance, AL-RF has the lowest mean best feasible incumbent among the tested screeners when all receive five final verifications. This result concerns shortlist quality; it does not include the 30 calibration calls.

In the five-seed comparison that counts all 35 calls, heuristic ranking has a slightly lower overall mean. Calibration helps on some instances, but this experiment does not show a consistent equal-call advantage for AL-RF. The method remains a candidate-screening option, not a replacement for a complete FJSP optimizer.

FUNDING

This work received no external funding.

CONFLICT OF INTEREST

The authors declare no conflict of interest.

DATA AVAILABILITY

Code, benchmark files, development summaries, and the five-seed records and protocol are available at https://github.com/thex1e2001-del/fjsp-active-learning. The original source revision is `8606fae`.

REFERENCES

[1] Adams, J., E. Balas, D. Zawack. 1988. The shifting bottleneck procedure for job shop scheduling. *Management Science* 34(3):391–401.

[2] Bengio, Y., A. Lodi, A. Prouvost. 2021. Machine learning for combinatorial optimization: A methodological tour d'horizon. *EJOR* 290(2):405–421.

[3] Bertsimas, D., J. Dunn. 2017. Optimal classification trees. *Machine Learning* 106:1039–1082.

[4] Brandimarte, P. 1993. Routing and scheduling in a flexible job shop by tabu search. *Annals of OR* 41(3):157–183.

[5] Breiman, L. 2001. Random forests. *Machine Learning* 45:5–32.

[6] Brucker, P., R. Schlie. 1990. Job-shop scheduling with multi-purpose machines. *Computing* 45(4):369–375.

[7] Calis, B., S. Bulkan. 2015. A research survey: review of AI solution techniques for job shop scheduling. *J. Intelligent Manufacturing* 26(5):961–973.

[8] Dauzère-Pérès, S., J. Paulli. 1997. An integrated approach for modeling and solving the general multiprocessor job-shop scheduling problem using tabu search. *Annals of OR* 70:281–306.

[9] Demšar, J. 2006. Statistical comparisons of classifiers over multiple data sets. *JMLR* 7:1–30.

[10] Elmachtoub, A.N., P. Grigas. 2022. Smart "predict, then optimize." *Management Science* 68(1):9–26.

[11] Fattahi, P., M. Saidi Mehrabad, F. Jolai. 2007. Mathematical modeling and heuristic approaches to flexible job shop scheduling problems. *J. Intelligent Manufacturing* 18:331–342.

[12] Gao, J., L. Sun, M. Gen. 2008. A hybrid genetic and variable neighborhood descent algorithm for flexible job shop scheduling problems. *Computers & OR* 35:2892–2907.

[13] Gasse, M., D. Chételat, N. Ferroni, L. Charlin, A. Lodi. 2019. Exact combinatorial optimization with graph convolutional neural networks. *NeurIPS* 32.

[14] He, H., H. Daumé III, J. Eisner. 2014. Learning to search in branch and bound algorithms. *NeurIPS* 27.

[15] Hurink, J., B. Jurisch, M. Thole. 1994. Tabu search for the job shop scheduling problem with multi-purpose machines. *OR Spektrum* 15:205–215.

[16] Hutter, F., H. Hoos, K. Leyton-Brown. 2011. Sequential model-based optimization for general algorithm configuration. *LION* 507–523.

[17] Jin, Y. 2011. Surrogate-assisted evolutionary computation. *Swarm and Evolutionary Computation* 1(2):61–70.

[18] Kacem, I., S. Hammadi, P. Borne. 2002. Approach by localization and multiobjective evolutionary optimization for FJSP. *IEEE Trans. SMC-C* 32(1):1–13.

[19] Khalil, E., P. Le Bodic, L. Song, G. Nemhauser, B. Dilkina. 2016. Learning to branch in mixed integer programming. *AAAI*.

[20] Kotary, J., F. Fioretto, P. Van Hentenryck, B. Wilder. 2021. End-to-end constrained optimization learning: A survey. *IJCAI*.

[21] Li, X., L. Gao. 2016. An effective hybrid genetic algorithm and tabu search for FJSP. *IJPE* 174:93–110.

[22] Maragno, D., H. Wiberg, D. Bertsimas, S.İ. Birbil, D. den Hertog, A. Fajemisin. 2025. Mixed-integer optimization with constraint learning. *Operations Research*.

[23] Mastrolilli, M., L.M. Gambardella. 2000. Effective neighborhood functions for the flexible job shop problem. *J. Scheduling* 3(1):3–20.

[24] Meng, L., C. Zhang, Y. Ren, B. Zhang, C. Lv. 2020. Mixed-integer linear programming and constraint programming formulations for solving distributed FJSP. *Computers & IE* 142:106347.

[25] Moon, S., S. Lee, K.-J. Park. 2024. Learning-enabled flexible job-shop scheduling for scalable smart manufacturing. *J. Manufacturing Systems* 77:356–367.

[26] Özgüven, C., L. Özbakır, Y. Yavuz. 2010. Mathematical models for job-shop scheduling. *Applied Math. Modelling* 34(6):1539–1548.

[27] Pezzella, F., G. Morganti, G. Ciaschetti. 2008. A genetic algorithm for the FJSP. *Computers & OR* 35:3202–3212.

[28] Pinedo, M. 2016. *Scheduling: Theory, Algorithms, and Systems*. 5th ed. Springer.

[29] Smit, I.G., Y. Wu, P. Troubil, Y. Zhang, W.P.M. Nuijten. 2025. Neural combinatorial optimization for stochastic FJSP. *AAAI*.

[30] Song, W., X. Chen, Q. Li, Z. Cao. 2023. Flexible job-shop scheduling via graph neural network and deep reinforcement learning. *IEEE Trans. Industrial Informatics* 19(2):1600–1610.

[31] Verwer, S., Y. Zhang, Q.C. Ye. 2017. Auction optimization using regression trees and linear models as integer programs. *Artificial Intelligence* 244:368–395.

[32] Wang, R., G. Wang, J. Sun, F. Deng, J. Chen. 2024. FJSP via dual attention network-based reinforcement learning. *IEEE Trans. Neural Networks and Learning Systems* 35:3091–3102.

[33] Xia, W., Z. Wu. 2005. An effective hybrid optimization approach for multi-objective FJSP. *Computers & IE* 48(2):409–425.

[34] Xie, J., L. Gao, K. Peng, X. Li, H. Li. 2019. Review on flexible job shop scheduling. *IET Collaborative Intelligent Manufacturing* 1(3):67–77.

[35] Zhang, C., W. Song, Z. Cao, J. Zhang, P.S. Tan, X. Chi. 2020. Learning to dispatch for job shop scheduling via deep reinforcement learning. *NeurIPS* 33:1621–1632.

[36] Zheng, X., L. Wang. 2016. A knowledge-guided fruit fly optimization algorithm for dual resource constrained FJSP. *IJPR* 54(18):5554–5566.